

Trabajo práctico: Memoria compartida

Consignas

Ejercicio 1: "Pienso un número"

El juego se llama "Pienso un Numero".

Se trata de un proceso PIENSO y otro JUGADOR (1 sólo jugador).

Ambos procesos utilizan MEMORIA COMPARTIDA:

```
struct juego
{
    char nombre_jugador[LARGO_NOMBRE];
    int numero_pensado.        // 0=indica sin número, del 1 a 99 número a verificar.
    int estado_acierto         // 0=No acierto, 1=acierto.
}
```

Además, utilizan un SEMÁFORO.

Proceso PIENSO: (MSG_PIENSO)

- 1) "Piensa un número", genera en forma aleatoria un número del 1 al 99 y lo guarda en una variable local `int local_pienso_un_numero`;
 - 2) Inicializa la MEMORIA COMPARTIDA y el SEMÁFORO.
 - 3) Espera el semáforo.
 - a) Si `numero_pensado <> 0` y `estado_acierto = 0`
 - i. Si `numero_pensado = local_pienso_un_numero`.
Actualiza `estado_acierto = 1`
 - ii. Si `numero_pensado <> local_pienso_un_numero`.
Actualiza `numero_pensado = 0` y `estado_acierto = 0`
 - 4) Libero el SEMÁFORO.
 - 5) Si hubo acierto, muestra por pantalla el nombre del jugador (sacarlo de MEMORIA COMPARTIDA) ganador y finaliza.
 - 6) Si no hubo acierto, regresa al punto 3.
-

Proceso JUGADOR:

Solicita por teclado el nombre del jugador y lo graba en MEMORIA COMPARTIDA.

- 1) Espera el semáforo.
- 2) Verifica de MEMORIA COMPARTIDA:
 - a) Si numero_pensado = 0 y estado_acierto = 0
 - i. Genera un número al azar de 1 al 99 (no repetitivos).
 - ii. Actualiza la variable numero_pensado con el número generado.
 - b) Si numero_pensado <> 0 y estado_acierto = 1
 - i. Acertó, muestra por pantalla la cantidad de intentos.
- 3) Liberar el semáforo.
- 4) Si no hubo acierto, repite desde el punto 1.
- 5) Si hubo acierto, finaliza.

Ejercicio 2: Simulación de peaje

Un PEAJE posee M vías (M es recibido por parámetro) representadas por un vector en memoria compartida.

intvehiculos_en_cola[CANTIDAD_DE_VIAS]; //cantidad de vehículos en cola por vía.

Además, existe otro proceso que simula a los autos: AUTOS

El proceso PEAJE atiende a una vía por vez cada N ms (N aleatorio entre 100 y 5000ms). Atiende siempre en un orden establecido VIA 1,2,3,4...M (y vuelve a empezar) restando un vehículo de la vía correspondiente. Si encuentra una vía con más de 10 vehículos en cola, deberá liberarla (pone la cantidad en 0).

El proceso PEAJE lleva un archivo con la cantidad de veces que tuvo que realizar una liberación (más de 10 vehículos en cola). Sólo con la cantidad de veces.

El proceso PEAJE muestra en pantalla en cada ciclo el total de vehículos en cola en cada vía:

VIA [1]: 5 Vehículos.

VIA [2]: 3 Vehículos.

VIA [3]: 3 Vehículos.

El proceso AUTOS verifica qué vía tiene menos vehículos y suma uno a la cantidad en la vía seleccionada. Esto lo repite cada N ms (N aleatorio entre 400 y 600ms).

EL proceso AUTOS debe mostrar por pantalla la vía seleccionada en cada caso.

Ambos procesos PEAJE y AUTOS reciben por parámetro la cantidad de vías disponibles (M).

Realizar la solución utilizando MEMORIA COMPARTIDA y SEMÁFAROS (siempre que sea necesario). En total son 2 procesos corriendo en forma simultánea. Se debe mostrar la actividad de cada proceso. El PEAJE, es quien inicializa el semáforo y los datos de inicio.

Ejercicio 3: Colchonera

Hay dos procesos: COLCHONERIA y VENDEDOR.

Estos procesos utilizan MEMORIA COMPARTIDA: 5 registros.

Struct descripción

```
{  
    Int código;  
    Int cantidad;  
    char nombre[LARGO_NOMBRE];  
}
```

Proceso COLCHONERÍA: (COLCHONERIA)

- 1) Carga la descripción de cada código por programa. La cantidad inicial es para todos 10;
 - 2) 1 PIERO, 2 SUAVESTAR, 3 CANNON, 4 SIMMONS, 5 BELMO
 - 3) Espera SEMÁFORO
 - 4) Revisa la cantidad de cada COLCHÓN; si es menor que 10, suma 1 al stock (SIEMPRE). NUNCA SE PASA DE 10.
 - 5) Libera SEMÁFORO.
 - 6) Espera 500ms.
 - 7) Repite desde el punto 2.
-

Proceso VENDEDOR:

- 1) Espera SEMÁFORO.
- 2) Solicita por TECLADO (OBLIGATORIO) un código y cantidad (1 a 5).
- 3) Resta esta cantidad al código seleccionado. NUNCA DEJA EL STOCK MENOR QUE 0.
- 4) Libera SEMÁFORO.

5) Vuelve al punto 1.